

Model Program Book



SEMESTER INTERNSHIP

Designed & Developed by



**ANDHRA PRADESH
STATE COUNCIL OF HIGHER EDUCATION**

(A STATUTORY BODY OF GOVERNMENT OF ANDHRA PRADESH)

PROGRAM BOOK FOR
SEMESTER INTERNSHIP

Name of the Student: **VALLURI GEETHA**

Name of the College: University College of Engineering Narasaraopet JNTUK

Registration Number: 23035A0509

Period of Internship: 4months From: 15-12-2025 To: 15-04-2026

Name & Address of the Intern Organization: Snestron Systems Pvt Ltd, Hyderabad.

**Jawaharlal Nehru Technological University
Kakinada**

YEAR
2025-2026

An Internship Report on
AI based Crypto Investment Manager

Submitted in accordance with the requirement for the degree of

Bachelor of Technology(B.Tech)

Under the Faculty Guideship of
Mrs.B.Jyothi

Department of Computer Science Engineering
University College of Engineering Narasaraopet JNTUK

Submitted by:
VALLURI GEETHA
Reg.No:23035A0509

Department of Computer Science Engineering

University College of Engineering Narasaraopet JNTUK

Student's Declaration

I, ValluriGeetha, a student of Bachelor of Technology Program, Reg.No. 23035A0509 of the Department of Computer Science and Engineering, University College of Engineering Narasaraopet JNTUK College do here by declare that I have Completed the mandatory internship from 15-12-2025 to 15-04-2026 in Snestron Systems Pvt.Ltd. under the Faculty Guide ship of Mrs.B.Jyothi, Department of Computer Science and Engineering, University College of Engineering Narasaraopet JNTUK.

(Signature and Date)

Official Certification

This is to certify that **VALLURI GEETHA** Reg. No. **23035A0509** has completed her Internship in **Snestron Systems Pvt Ltd** on **AI based Crypto Investment Manager** under my supervision as a part of partial fulfillment of the requirement for the Degree of **Bachelor of Technology (B.Tech)** in the Department of **Computer Science Engineering University College of Engineering Narasaraopet JNTUK**.

This is accepted for evaluation.

(Signatory with Date and Seal)

Endorsements

Faculty Guide

Head of Department

Certificate from Internship Organization



SNESTRON SYSTEMS PRIVATE LIMITED

8-7-7/2, Plot NO. 51, Opp: Naveena School, Hasthinapuram Central, Hyderabad,
500 079. Telangana



CERTIFICATE OF INTERNSHIP EXPERIENCE

To whom so ever it may concern

This is to certify that **Valluri Geetha** with Intern id: **ST40F297** has successfully completed a **4 months** Online Internship Program in the domain of **Artificial Intelligence**. in **SNESTRON SYSTEMS PRIVATE LIMITED**. from **December 15th, 2025 to April 15th, 2026**.

During the internship, His/Her demonstrated outstanding dedication, creativity, and technical proficiency. His/Her performance in the assigned projects was exceptional, showcasing deep understanding, innovative problem-solving skills, and a strong commitment to excellence.

We appreciate his/her active participation, consistent efforts, and impressive contribution to the overall success of the internship. We wish him/her all the best in future endeavors.

We are confident that his/her dedication and skills will lead to great achievements ahead.

Best Wishes,
KAMMAMPATI SATISH
HR MANAGER

HR@SNESTRONSYSYSTEMS.COM



#startupindia

Acknowledgements

It is needed with a great sense of pleasure and immense sense of gratitude that we acknowledge the help of these individuals. We owe many thanks to many people who helped and supported us during the writing of this report.

We take privilege to express our heartfelt gratitude to Project Supervisor, **Mrs.B.Jyothi**, Assistant professor(c) of Department of CSE and Head of the Department(i/c) **Dr.G.Madhavi** for the valuable suggestions and constant motivation that greatly helped us in successful completion of project. We express our sincere thanks to **Dr.Y.S.Kishore Babu**, Vice Principal and **Prof.Ch.Srinivasa Rao**, Principal, University College of Engineering Narasaraopet-JNTUK, for providing us with a good infrastructure and environment to carry out this project.

We are thankful to all faculty members for extending their kind cooperation and assistance. Finally, we are extremely thankful to our parents and friends for their constant help and moral support.

V.Geetha

23035A0509

LIST OF CONTENTS

S.NO	CONTENTS	PG.NO
1	ABSTRACT	10-11
2	INTRODUCTION	12
	2.1 BACKGROUND	13
	2.2 ADVANTAGES	13
	2.3 LIMITATIONS	13
	2.4 IMPLEMENTATION	13
	2.5 DEPLOYMENT AND VISUALIZATION	13
	2.6 OBJECTIVES	13
3	PROJECT OVERVIEW	14
	3.1 PROJECT OBJECTIVES	15
	3.2 REQUIREMENTS AND PLANNING	15-16
	3.3 ENVIRONMENT SETUP	16
	3.4 DATABASE DESIGN	16
	3.5 DELIVERABLES	16
4	TECHNOLOGIES	17
	4.1.PYTHON LIBRARIES	18
	4.2 FRONTEND TECHNOLOGIES	18
	4.3 API INTEGRATION	18
	4.4 DATAPROCESSING TECHNIQUES	18
5	FEATURES USED IN AI BASED CRYPTO INVESTMENT MANAGER	19
	5.1 FEATURES	20
	5.2 IMPORTANCE OF FEATURES	20
	5.3 CORE FUNCTIONALITY	21
6	DEVELOPMENT PROCESS	22-23
7	METHODOLOGY	24
	7.1 DATA LOADING AND PREPROCESSING	25
	7.2 MODEL DEVELOPMENT AND TRAINING	25-26
	7.3 EXPLORATORY DATA ANALYSIS	27-31
	7.4 BACKEND/API DEVELOPMENT	32-36
	7.5 SYSTEM OUTPUT:WEB APPLICATION INTERFACE	37
8	DATA FLOW DIAGRAMS	38

	8.1Level0 DFD	39
	8.2Level1 DFD	40
9	FUTURE ENHANCEMENTS	41-42
10	RESULTS AND ACHIEVEMENTS	43-44
11	CONCLUSION	45-46
12	REFERENCES	47-48

ABSTRACT

1.ABSTRACT

The rapid expansion of cryptocurrency markets has attracted a large number of investors due to their high return potential. However, the cryptocurrency market is highly volatile, dynamic, and unpredictable, which makes investment decisions challenging for both beginners and experienced investors. Traditional investment approaches often rely on manual analysis and intuition, which may not always produce reliable results. To address these challenges, the **AI-Based Crypto Investment Manager** project is developed to assist investors in analyzing cryptocurrency data and making informed investment decisions using data analytics and artificial intelligence techniques.

The system collects historical market data of various cryptocurrencies and performs data processing and analysis to evaluate key financial indicators such as price trends, volatility levels, and risk factors. By integrating artificial intelligence and machine learning concepts, the system can identify patterns in market behavior and provide insights that help users understand potential investment opportunities and risks. The application uses Python and data analysis libraries to efficiently process large datasets and perform statistical and predictive analysis.

In addition to risk analysis, the system provides a structured representation of investment insights, enabling users to better interpret cryptocurrency performance and market fluctuations. Visualization techniques can also be used to present trends and comparisons among different cryptocurrencies. This approach helps investors reduce uncertainty and supports data-driven decision-making rather than relying purely on speculation.

To overcome these challenges, advanced technologies such as data analytics and artificial intelligence can be used to analyze large volumes of cryptocurrency market data and identify patterns that may not be easily visible through traditional analysis methods. Artificial intelligence techniques can help evaluate historical data, detect trends, measure volatility, and provide insights that assist investors in understanding potential risks and opportunities.

Overall, the **AI-Based Crypto Investment Manager** demonstrates how artificial intelligence, financial analytics, and data visualization techniques can be combined to create a practical tool for cryptocurrency market analysis, investment risk assessment, and improved strategic decision-making.

INTRODUCTION

2.INTRODUCTION

2.1 BACKGROUND

The **AI-Based Crypto Investment Manager** project is designed to support investors by providing a system that analyzes cryptocurrency market data and generates meaningful insights regarding market trends and investment risks. The system uses Python and data analysis tools to collect and process cryptocurrency data, perform risk analysis, and present the results in a structured and understandable manner. By leveraging artificial intelligence and data-driven analysis, the system aims to help users make more informed and strategic investment decisions in the dynamic cryptocurrency market.

2.2 ADVANTAGES

- Helps investors make better and informed investment decisions.
- Provides risk analysis by analyzing cryptocurrency market volatility.
- Saves time and effort by automating data analysis.
- Uses Artificial Intelligence to identify patterns and trends in the market.
- Helps users understand cryptocurrency market behavior more clearly.
- Presents data through visualizations such as charts and graphs for easy understanding.
- Useful for both beginners and experienced investors.
- Reduces human errors that may occur in manual analysis.
- Allows easy analysis of multiple cryptocurrencies.
- Can be expanded with advanced AI models and real-time data in the future.

2.3 LIMITATIONS

- Market volatility makes it difficult to predict cryptocurrency prices accurately.
- The system depends on historical data, which may not always reflect future market behavior.
- AI predictions are not always 100% accurate, especially in highly unpredictable markets.
- The system may not consider external factors such as government regulations, global events, or sudden market news.
- Requires continuous data updates to provide accurate analysis.
- Limited to the cryptocurrencies included in the dataset.

2.4 IMPLEMENTATION

The AI-Based Crypto Investment Manager is implemented using Python and various data analysis libraries to analyze cryptocurrency market data and provide insights for investment decisions. The implementation process involves several stages including data collection, data preprocessing, analysis, and result visualization.

2.5 DEPLOYMENT AND VISUALIZATION

The AI-Based Crypto Investment Manager is deployed as a Python-based application that allows users to analyze cryptocurrency data and view investment insights. The system is developed and executed in a Python environment using tools such as Jupyter Notebook or an integrated development environment (IDE) like VS Code.

Data visualization plays an important role in understanding cryptocurrency market behavior. In the AI-Based Crypto Investment Manager, visualization techniques are used to represent cryptocurrency data in a graphical and easy-to-understand format.

2.6 OBJECTIVE

- To analyze cryptocurrency market data using data analytics techniques.
- To assist investors in making informed investment decisions.
- To evaluate the risk and volatility associated with different cryptocurrencies.
- To apply artificial intelligence techniques for identifying patterns and trends in crypto markets.

PROJECT OVERVIEW

3.PROJECT OVERVIEW

The **AI-Based Crypto Investment Manager** is a data-driven system designed to assist investors in analyzing and managing cryptocurrency investments. The cryptocurrency market is highly volatile and unpredictable, making it challenging for investors to make informed decisions based solely on intuition or limited market knowledge. This project addresses this problem by combining **artificial intelligence (AI)**, **data analytics**, and **visualization techniques** to provide meaningful insights into cryptocurrency trends and risks.

3.1.PROJECT OBJECTIVES

- To evaluate investment risks and provide insights on market volatility.
- To apply machine learning and AI techniques to identify patterns and trends in the market.
- To provide visual representations of market trends and cryptocurrency performance.
- To reduce reliance on speculation by supporting strategic investment decisions.
- To create a user-friendly platform for cryptocurrency analysis and investment guidance.
- To enhance investment efficiency by automating data processing and analysis.
- To demonstrate the integration of AI, data analytics, and visualization in financial technology applications.

3.2.REQUIREMNET ANALYSIS AND PLANNING

- **Data Requirements:** The system requires historical and real-time cryptocurrency data, including price, trading volume, market trends, and other financial indicators, in a structured format for AI-based analysis.
- **FunctionalRequirements:**
 - Ability to fetch historical and real-time cryptocurrency data (price, volume, market trends).
 - Cleaning and organizing raw data, handling missing values, and converting data into a structured format suitable for analysis.
 - Calculating volatility, returns, and other risk factors for different cryptocurrencies.
 - Applying machine learning and AI techniques to detect patterns, trends, and provide predictive insights.
- **Non-FunctionalRequirements:**
 - The system should process and analyze large cryptocurrency datasets efficiently.
 - The system should be scalable to accommodate additional cryptocurrencies or larger datasets in the future..
 - AI-based predictions should be as accurate as possible, based on historical data and trends.
- **Planning:**
 - Requirement Gathering: Identify what the system must achieve for investors.
 - System Design: Plan the architecture, including data collection, AI analysis, visualization, and interface.
 - Data Acquisition and Preprocessing: Collect cryptocurrency data and clean it for analysis.
 - AI Model Integration: Apply AI/ML techniques for pattern recognition and predictive analysis.

- **Visualization & Reporting:** Develop graphs, charts, and reports to present insights clearly.
- **Testing & Validation:** Ensure analysis results are accurate and the system is reliable.
- **Deployment:** Make the system available locally or online for practical use.

3.3. ENVIRONMENT SETUP

- **ProgrammingLanguage:**Python
- **DevelopmentEnvironment:** VS Code
- **Libraries&Frameworks:**
 - **DataAnalysis:**Pandas,NumPy
 - **Visualization:**Matplotlib
 - **MachineLearning:**Scikit-learn
 - **WebDevelopment:**Streamlit
 - **APITesting:**CoinGeckoAPI
- **FrontendTechnologies:**HTML,CSS, JavaScript

3.4.DATABASE DESIGN

- **DataSource:** CSV files can be used for simpler implementations, especially for historical market data.
- **DataHandling:**Data was loaded,transformed,and used directly in-memory using Pandas DataFrames.
- Historical and real-time cryptocurrency data is stored in the Market Data Table.
- AI and analysis modules fetch data from the database, perform computations, and store results or predictions back in structured tables.
- Visualization modules retrieve processed data to generate charts and reports for users.

3.5.DELIVERABLES

The **AI-Based Crypto Investment Manager** project provides the following key deliverables:

1. **Historical and Real-Time Data Analysis**
 - Processed and structured cryptocurrency market data including price, volume, and trends.
2. **Risk Assessment Reports**
 - Evaluation of volatility, returns, and potential risks associated with different cryptocurrencies.
3. **AI-Based Predictions and Insights**
 - Pattern recognition, trend analysis, and predictive insights to support investment decisions.
4. **Data Visualization**
 - Graphs, charts, and tables displaying market trends, fluctuations, and comparisons among cryptocurrencies.

TECHNOLOGIES USED

4. Technologies Used in AI based Crypto Investment Manager

4.1. PYTHON LIBRARIES

Python is the primary programming language used for this project due to its extensive libraries and community support in data science.

- **Pandas:** Used for data manipulation and analysis. It provides data structures like DataFrames that make handling tabular data easy, enabling cleaning, filtering, and transforming real estate datasets.
- **NumPy:** A fundamental package for scientific computing in Python, used for efficient numerical operations, array handling, and mathematical functions. It supports fast computations essential for preprocessing and modeling.
- **Scikit-learn:** This powerful machine learning library was used for model building, including training the Linear Regression model. It offers simple APIs for data preprocessing, feature engineering, model selection, evaluation metrics, and tuning techniques such as Grid Search and K-Fold Cross Validation.
- **Matplotlib:** A widely used plotting library for visualizing data trends, distributions, and model results. It helped in generating graphs that illustrate price distributions, correlations, and prediction accuracy.

4.2. FRONTEND TECHNOLOGIES

- **Streamlit:** Streamlit is used to build an interactive user interface for the application where users can select cryptocurrencies, view analysis results, and visualize predictions.

4.3. API INTEGRATION

- **API Used:** CoinGecko API
The API is used to fetch real-time and historical cryptocurrency market data.

4.4. DATA PROCESSING AND TECHNIQUES

- Historical and real-time cryptocurrency data is collected using CoinGecko API.
- Data includes cryptocurrency prices, trading volumes, market capitalization, and historical trends.
- Scaling or standardizing numeric values (e.g., prices, volume) to make them comparable across different cryptocurrencies.
- Calculating additional metrics like daily returns, volatility, and moving averages for risk assessment and trend analysis.
- Organizing processed data for use in Streamlit dashboards and generating charts with Matplotlib/Seaborn.

FEATURES

5.FEATURES USED IN AI BASED CRYPTPO INVESTMENT MANAGER

5.1.FEATURES

The **AI-Based Crypto Investment Manager** offers several features to help users analyze cryptocurrency markets and make informed investment decisions:

1. **Real-Time Data Fetching**
 - Uses **CoinGecko API** to collect live cryptocurrency prices, trading volumes, and market trends.
2. **Historical Data Analysis**
 - Processes past cryptocurrency data to identify trends, calculate returns, and assess market behavior.
3. **AI-Based Predictions**
 - Applies AI and machine learning techniques to detect patterns and provide predictive insights on cryptocurrency price movements.
4. **Risk Assessment**
 - Calculates market volatility, daily returns, and other risk metrics to help investors evaluate potential investment risks.
5. **Data Visualization**
 - Displays data in **interactive charts and graphs** using Streamlit, Matplotlib, and Seaborn for easier interpretation.
6. **User-Friendly Interface**
 - Provides an interactive **Streamlit dashboard** that allows users to select cryptocurrencies, view trends, and analyze data easily.
7. **Comparison of Cryptocurrencies**
 - Allows users to compare multiple cryptocurrencies in terms of price trends, volatility, and market capitalization.
8. **Customizable Time Frames**
 - Users can view data over different time periods (daily, weekly, monthly) for better analysis.
9. **Data Export (Optional)**
 - Provides an option to export processed data and analysis results for offline use or reporting.
10. **Scalable System**
 - The system can be extended to include more cryptocurrencies, advanced AI models, or real-time trading alerts in future enhancements.

5.2.IMPORTANCE OF FEATURES

The features of the **AI-Based Crypto Investment Manager** play a crucial role in enhancing investment decision-making and improving user experience. Real-time data fetching ensures that investors have access to the latest market information, which is essential in the highly volatile cryptocurrency market. Historical data analysis and AI-based predictions help identify trends and patterns, allowing users to make informed and strategic investment decisions rather than relying solely on speculation. Risk assessment provides insights into potential market volatility and financial risks, while data visualization converts complex datasets into intuitive charts and graphs for easier interpretation. The user-friendly Streamlit interface, customizable time frames, and the ability to compare multiple cryptocurrencies make the system accessible and flexible for both beginners and experienced investors.

5.3.CORE FUNCTIONALITIES OF AI BASED CRYPTO INVESTMENT MANAGER

Core Functionalities

1. **Data Acquisition** – Fetches historical and real-time cryptocurrency data using the **CoinGecko API**.
2. **Data Preprocessing** – Cleans, organizes, and transforms raw data into structured formats for analysis.
3. **Data Analysis** – Calculates key financial metrics such as returns, volatility, and moving averages.
4. **AI-Based Predictive Analysis** – Uses AI/ML techniques to identify market patterns and trends.
5. **Risk Assessment** – Evaluates the potential risk of investments based on volatility and historical trends.
6. **Interactive Visualization** – Displays charts, graphs, and dashboards using **Streamlit**, **Matplotlib**, and **Seaborn**.
7. **Cryptocurrency Comparison** – Allows users to compare multiple cryptocurrencies based on performance metrics.
8. **Customizable Time Frames** – Enables analysis over different periods (daily, weekly, monthly) for better insights.
9. **Data Export** – Provides the option to export processed data and reports for offline use.
10. **User-Friendly Interface** – Offers an intuitive platform accessible to both beginners and experienced investors.

DEVELOPMENT PROCESS

6.DEVELOPMENT PROCESS OF AI BASED CRYPTO INVESTMENT MANAGER

Development Process

- 1. Requirement Analysis**
 - Identified project objectives, functional and non-functional requirements, and user needs for cryptocurrency investment analysis.
- 2. Planning and Design**
 - Designed system architecture, database structure, and Streamlit-based user interface.
 - Planned the integration of CoinGecko API for real-time data fetching and AI modules for predictive analysis.
- 3. Data Collection**
 - Gathered historical and real-time cryptocurrency data from CoinGecko API.
 - Stored data in a structured format (CSV or SQLite database) for analysis.
- 4. Data Preprocessing**
 - Cleaned datasets by handling missing values and duplicates.
 - Normalized and transformed data for consistent and efficient analysis.
- 5. AI/ML Model Integration**
 - Applied machine learning or AI techniques to identify trends, patterns, and predictive insights in cryptocurrency markets.
- 6. Data Analysis**
 - Calculated key financial metrics like returns, volatility, and moving averages to support risk assessment.
- 7. Visualization and Dashboard Development**
 - Developed interactive charts, graphs, and dashboards using Streamlit and Matplotlib.
 - Enabled users to view insights, compare cryptocurrencies, and analyze trends.
- 8. Testing and Validation**
 - Tested the system for accuracy of analysis, reliability of AI predictions, and correctness of visualizations.
- 9. Deployment**
 - Deployed the system as a Python-based application accessible via Streamlit interface.
 - Ensured smooth execution and user-friendly experience for investors.
- 10. Documentation**
 - Prepared project report, including objectives, features, functionalities, database design, and development methodology.

METHODOLOGY

7.METHODOLOGY

7.1.DATA LOADING AND PREPROCESSING

- Data Loading – Collected 30-days historical cryptocurrency data using the CoinGecko API and loaded it into Pandas DataFrames for processing.
- Handling Missing Values – Checked for missing or null entries in the dataset and filled or removed them to ensure data accuracy.
- Removing Duplicates – Eliminated duplicate records to maintain consistency.
- Data Transformation – Converted raw API data into a structured format suitable for analysis (e.g., CSV or DataFrame).
- Normalization/Scaling – Standardized numeric values such as prices, market capitalization, and trading volumes for better comparison.
- **Market snapshot.csv**

```
coin_id,name,current_price,market_cap
bitcoin,Bitcoin,70500,1412317535415
ethereum,Ethereum,2058.13,248816339135
tether,Tether,0.999979,183923077422
binancecoin,BNB,645.6,88030597606
ripple,XRP,1.39,85207477535
usd-coin,USDC,0.999906,78195263514
solana,Solana,86.7,49495051181
tron,TRON,0.28417,26920627076
figure-heloc,Figure Heloc,1.042,16302598332
dogecoin,Dogecoin,0.09455,14489470732
whitebit,WhiteBIT Coin,55.98,11963576734
usds,USDS,0.999976,11123199286
cardano,Cardano,0.264138,9741219047
bitcoin-cash,Bitcoin Cash,446.52,8940106025
leo-token,LEO Token,9.16,8435844744
hyperliquid,Hyperliquid,34.89,8323918121
chainlink,Chainlink,9.07,6425844165
monero,Monero,345.49,6379674534
ethena-usde,Ethena USDe,0.999318,5921188682
canton-network,Canton,0.148693,5661039877
stellar,Stellar,0.160582,5302651982
usd1-wlfi,USD1,0.999409,4607244342
dai,Dai,1.0,4262885400
rain,Rain,0.00885538,4237197848
hedera-hashgraph,Hedera,0.096351,4177440189
```

7.2.MODEL DEVELOPMENT AND TRAINING

1. Data Preparation

- Used the **30-days historical cryptocurrency data** collected from **CoinGecko API**.
- Processed the data by cleaning, normalizing, and extracting features such as **daily returns, price volatility, and moving averages**.
- Converted the dataset into a format suitable for model training (e.g., time-series sequences).

- **Collect.py**

```
1 import requests
2 import csv
3
4 API_URL = "https://api.coingecko.com/api/v3/coins/markets"
5 SNAPSHOT_CSV = "market_snapshot.csv"
6
7 def fetch_market_data():
8     params = {
9         "vs_currency": "usd",
10        "order": "market_cap_desc",
11        "per_page": 50,
12        "page": 1,
13        "sparkline": "false"
14    }
15
16    response = requests.get(API_URL, params=params)
17    response.raise_for_status()
18    return response.json()
19
20 def save_to_csv(data):
21     with open(SNAPSHOT_CSV, mode="w", newline="", encoding="utf-8") as file:
22         writer = csv.writer(file)
23         writer.writerow(["coin_id", "name", "current_price", "market_cap"])
24
25         for coin in data:
26             writer.writerow([
27                 coin["id"],
28                 coin["name"],
29                 coin["current_price"],
30                 coin["market_cap"]
31             ])
32
33 def main():
34     print("Fetching market data...")
35     data = fetch_market_data()
36     save_to_csv(data)
37     print("Market data saved to market_snapshot.csv")
38
39 if __name__ == "__main__":
40     main()
```

7.3.EXPLORATORY DATA ANALYSIS (EDA)

1. Feature Selection

- Selected relevant features that impact cryptocurrency price trends, including closing price, trading volume, and historical volatility.
- Ensured features are scaled and normalized for better model performance.

2. Model Selection

- Chose a machine learning or AI approach suitable for time-series analysis, such as Linear Regression for predicting trends.

3. Training the Model

- Split the dataset into training and testing sets (e.g., 80% training, 20% testing).
- Trained the selected model on the historical data to learn patterns and trends in cryptocurrency prices.

4. Evaluation

- Evaluated model performance using metrics such as Mean Absolute Error (MAE), Mean Squared Error (MSE), or Root Mean Squared Error (RMSE).
- Adjusted model parameters to improve prediction accuracy.

5. Prediction and Insights

- Used the trained model to predict short-term trends and provide insights into potential market behavior.
- Integrated predictions into the Streamlit dashboard for visualization and decision-making support.

6. Model Update

- The model can be retrained periodically with new historical or real-time data to improve accuracy and adapt to market changes

- Eda_portfolio.py

```

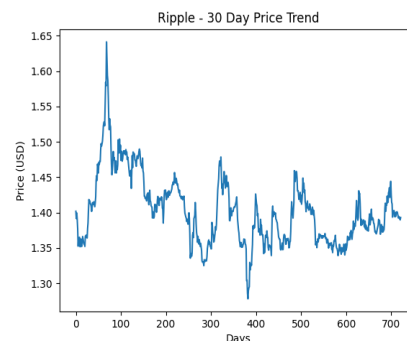
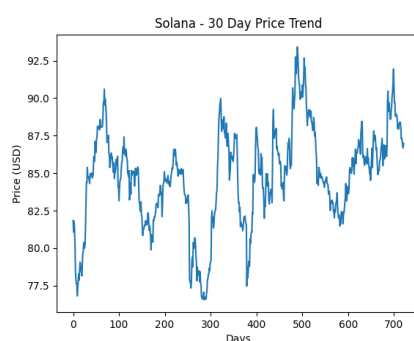
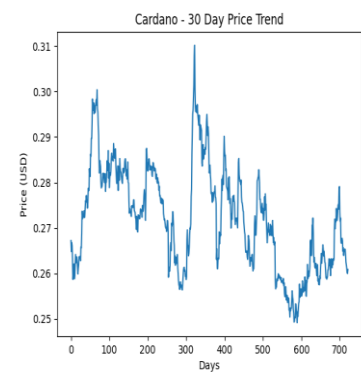
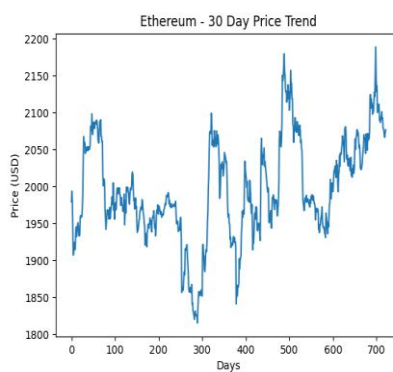
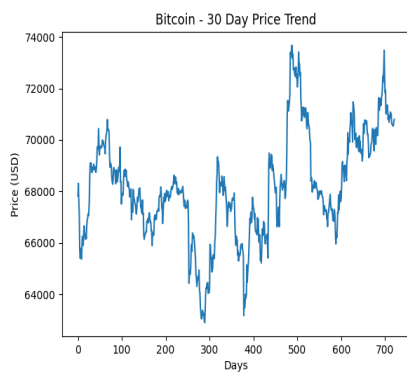
1  # -----
2  # EDA PORTFOLIO ANALYSIS WITH VISUALIZATION + FINANCIAL METRICS
3  # -----
4
5  import requests
6  import statistics
7  import time
8  import matplotlib.pyplot as plt
9  from prediction import predict_next_price
10 TOTAL_INVESTMENT = 10000
11 RISK_FREE_RATE = 0.01
12
13 COINS = ["bitcoin", "ethereum", "cardano", "solana", "ripple"]
14
15
16 # -----
17 # STEP 1: Fetch 30-Day Historical Prices
18 # -----
19
20 def fetch_historical_prices(coin_id):
21
22     url = f"https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart"
23
24     params = {
25         "vs_currency": "usd",
26         "days": 30
27     }
28
29     response = requests.get(url, params=params)
30     response.raise_for_status()
31
32     data = response.json()
33
34     prices = [price_point[1] for price_point in data["prices"]]
35
36     return prices
37
38
39 # -----
40 # STEP 2: Calculate Returns
41 # -----
42
43 def calculate_returns(prices):
44
45     returns = []
46
47     for i in range(1, len(prices)):
48         r = (prices[i] - prices[i-1]) / prices[i-1]
49         returns.append(r)
50
51     return returns
52
53
54 # -----
55 # STEP 3: Expected Return
56 # -----
57
58 def calculate_expected_return(returns):
59
60     if len(returns) > 0:
61         return statistics.mean(returns)
62
63     return 0
64
65
66 # -----
67 # STEP 4: Volatility (Risk)
68 # -----
69
70 def calculate_volatility(returns):
71
72     if len(returns) > 1:
73         return statistics.stdev(returns)
74
75     return 0
76
77
78 # -----
79 # STEP 5: Sharpe Ratio
80 # -----
81
82 def calculate_sharpe_ratio(expected_return, volatility):
83
84     if volatility == 0:
85         return 0
86
87     sharpe = (expected_return - RISK_FREE_RATE) / volatility
88     return sharpe
89
90
91 # -----
92 # STEP 6: Risk Classification
93 # -----
94
95 def classify_risk(volatility_data):
96
97     sorted_coins = sorted(volatility_data.items(), key=lambda x: x[1])
98
99     n = len(sorted_coins)
100
101     risk_groups = {
102         "low": [],
103         "medium": [],
104         "high": []
105     }
106
107     for i, (coin, vol) in enumerate(sorted_coins):
108
109         if i < n/3:
110             risk_groups["low"].append(coin)
111
112         elif i < 2*n/3:
113             risk_groups["medium"].append(coin)
114
115         else:
116             risk_groups["high"].append(coin)
117
118     return risk_groups
119
120
121 # -----
122 # STEP 7: Investment Allocation
123 # -----
124
125 def allocate_investment(risk_groups):
126
127     allocation_ratio = {
128         "low": 0.5,
129         "medium": 0.3,
130         "high": 0.2
131     }
132
133     final_allocation = {}
134
135     for level, coins in risk_groups.items():
136
137         if coins:
138
139             amount_per_coin = (TOTAL_INVESTMENT * allocation_ratio[level]) / len(coins)
140
141             for coin in coins:
142                 final_allocation[coin] = round(amount_per_coin, 2)
143
144     return final_allocation

```

```

1 # -----
2 # VISUALIZATION FUNCTIONS
3 # -----
4
5 def plot_price_trend(coin, prices):
6
7     plt.figure()
8     plt.plot(prices)
9     plt.title(f"{coin.capitalize()} - 30 Day Price Trend")
10    plt.xlabel("Days")
11    plt.ylabel("Price (USD)")
12    plt.show()
13
14
15 def plot_volatility(volatility_data):
16
17     plt.figure()
18     coins = list(volatility_data.keys())
19     values = list(volatility_data.values())
20     plt.bar(coins, values)
21     plt.title("Volatility Comparison")
22     plt.xlabel("Coins")
23     plt.ylabel("Volatility")
24     plt.xticks(rotation=45)
25     plt.show()
26
27
28 def plot_allocation(final_mix):
29
30     plt.figure()
31     coins = list(final_mix.keys())
32     amounts = list(final_mix.values())
33     plt.pie(amounts, labels=coins, autopct="%1.1f%%")
34     plt.title("₹10,000 Portfolio Allocation")
35     plt.show()
36
37 def plot_prediction(prices, predicted_price):
38
39     X = list(range(len(prices)))
40     plt.plot(X, prices, label="Actual Prices")
41     plt.scatter(len(prices), predicted_price, color="red", label="Predicted Price")
42     plt.title("Price Prediction")
43     plt.xlabel("Days")
44     plt.ylabel("Price")
45     plt.legend()
46     plt.show()

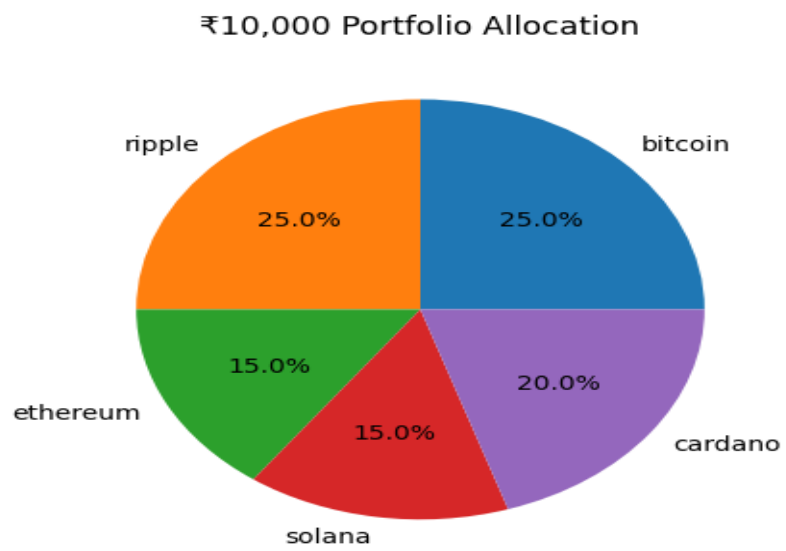
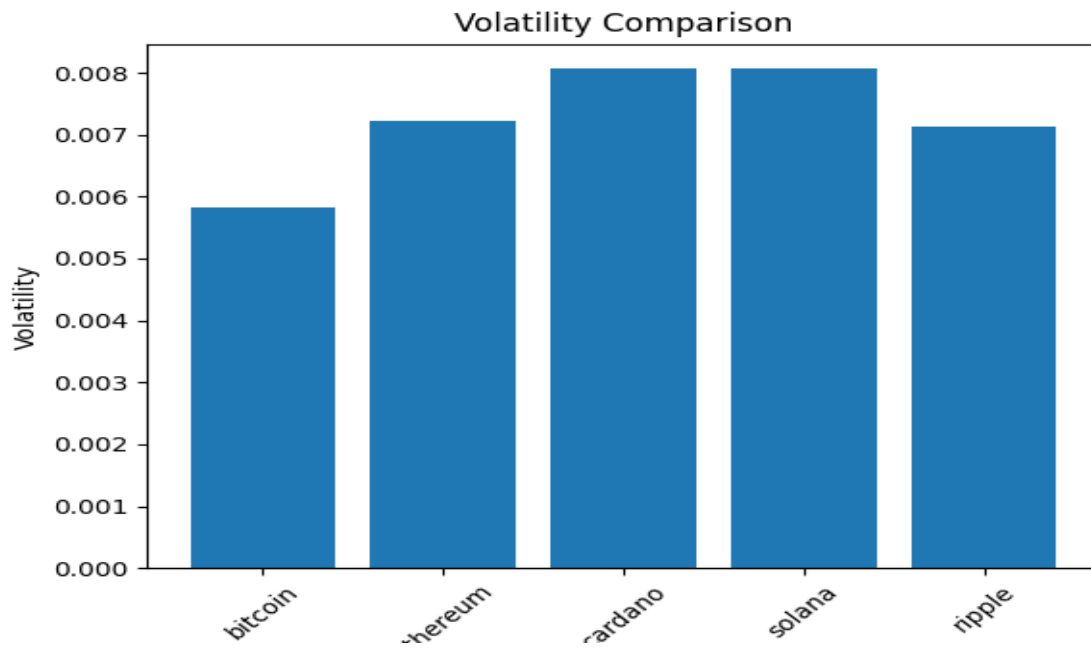
```



```

1  # -----
2  # MAIN EXECUTION
3  # -----
4
5  def main():
6
7      volatility_data = {}
8      expected_returns = {}
9      sharpe_ratios = {}
10     print("----- EDA & PORTFOLIO ANALYSIS ----- \n")
11
12     for coin in COINS:
13
14         try:
15
16             prices = fetch_historical_prices(coin)
17             plot_price_trend(coin, prices)
18             returns = calculate_returns(prices)
19             exp_return = calculate_expected_return(returns)
20             volatility = calculate_volatility(returns)
21             sharpe = calculate_sharpe_ratio(exp_return, volatility)
22             predicted_price = predict_next_price(prices)
23             plot_prediction(prices, predicted_price)
24             volatility_data[coin] = volatility
25             expected_returns[coin] = exp_return
26             sharpe_ratios[coin] = sharpe
27
28             print(f"{coin.upper()}")
29             print("Expected Return:", round(exp_return,6))
30             print("Volatility:", round(volatility,6))
31             print("Sharpe Ratio:", round(sharpe,4))
32             print("Predicted Next Price:", round(predicted_price,2))
33             print("-----")
34
35             time.sleep(2)
36
37         except Exception as e:
38             print(f"Error for {coin}: {e}")
39             time.sleep(5)
40
41     if not volatility_data:
42         print("No data available.")
43         return
44
45     # Plot volatility comparison
46     plot_volatility(volatility_data)
47     # Risk classification
48     risk_groups = classify_risk(volatility_data)
49
50     print("\nRisk Classification:")
51
52     for level, coins in risk_groups.items():
53         print(level.upper(), "RISK →", coins)
54
55     # Portfolio allocation
56     final_mix = allocate_investment(risk_groups)
57
58     print("\nPortfolio Allocation (₹10,000):")
59
60     for coin, amount in final_mix.items():
61
62         weight = (amount / TOTAL_INVESTMENT) * 100
63         print(f"{coin} → ₹{amount} | Weight: {round(weight,2)}%")
64
65     # Pie chart
66     plot_allocation(final_mix)
67     print("\n----- ANALYSIS COMPLETE -----")
68
69 if __name__ == "__main__":
70     main()

```



7.4. BACKEND / API DEVELOPMENT

- The fetched data is processed in the backend to calculate daily returns, expected return, and volatility.
- Based on the calculated volatility, each cryptocurrency is classified into Low Risk, Medium Risk, or High Risk categories.
- The backend module then generates a detailed analysis report containing risk levels and performance metrics.
- The final results are automatically saved into a text-based report file (crypto_report.txt) for future reference.
- **Report.py**

```
1  # -----
2  # CRYPTO ANALYSIS REPORT
3  # -----
4
5  from risk import (
6      fetch_crypto_prices,
7      calculate_returns,
8      calculate_volatility,
9      calculate_expected_return,
10     classify_risk
11 )
12
13 import time
14
15 COINS = ["bitcoin", "ethereum", "cardano", "solana", "ripple"]
16
17
18 # -----
19 # Generate Report
20 # -----
21
22 def generate_report():
23
24     report_file = open("crypto_report.txt", "w")
25
26     print("\n===== CRYPTO PORTFOLIO REPORT =====\n")
27     report_file.write("===== CRYPTO PORTFOLIO REPORT =====\n\n")
28
29     for coin in COINS:
30
31         print("Analyzing:", coin.capitalize())
32         report_file.write(f"Analyzing: {coin.capitalize()}\n")
33
34         prices = fetch_crypto_prices(coin)
35
36         if not prices:
37             print("No data available\n")
38             report_file.write("No data available\n\n")
39             continue
40
41         returns = calculate_returns(prices)
42
43         expected_return = calculate_expected_return(returns)
44
45         volatility = calculate_volatility(returns)
46
47         risk = classify_risk(volatility)
48
49         print("Expected Return:", round(expected_return, 6))
50         print("Volatility:", round(volatility, 6))
51         print("Risk Level:", risk)
52         print("-----\n")
53
54         report_file.write(f"Expected Return: {round(expected_return,6)}\n")
55         report_file.write(f"Volatility: {round(volatility,6)}\n")
56         report_file.write(f"Risk Level: {risk}\n")
57         report_file.write("-----\n\n")
58
59         time.sleep(2)
60
61     print("===== REPORT COMPLETE =====")
62     report_file.write("===== REPORT COMPLETE =====")
63
64     report_file.close()
65
66
67 # -----
68
69 if __name__ == "__main__":
70     generate_report()
```


Crypto_report.txt



```
1  ===== CRYPTO PORTFOLIO REPORT =====
2
3  Analyzing: Bitcoin
4  Expected Return: 4e-06
5  Volatility: 0.005923
6  Risk Level: Medium Risk
7  -----
8
9  Analyzing: Ethereum
10 Expected Return: -1.1e-05
11 Volatility: 0.007258
12 Risk Level: Medium Risk
13 -----
14
15 Analyzing: Cardano
16 Expected Return: -2.9e-05
17 Volatility: 0.008065
18 Risk Level: Medium Risk
19 -----
20
21 Analyzing: Solana
22 Expected Return: 2.7e-05
23 Volatility: 0.008029
24 Risk Level: Medium Risk
25 -----
26
27 Analyzing: Ripple
28 Expected Return: -1.5e-05
29 Volatility: 0.007302
30 Risk Level: Medium Risk
31 -----
32
33 ===== REPORT COMPLETE =====
```

- risk.py

```
1  # -----
2  # Risk Classification
3  # -----
4
5  def classify_risk(volatility):
6
7      if volatility < 0.005:
8          return "Low Risk"
9
10     elif volatility < 0.01:
11         return "Medium Risk"
12
13     else:
14         return "High Risk"
15
16
17  # -----
18  # Main Risk Analysis
19  # -----
20
21  def main():
22
23      print("\n----- CRYPTO RISK ANALYSIS ----- \n")
24
25      for coin in COINS:
26
27          print("Analyzing", coin.capitalize(), "...")
28
29          prices = fetch_crypto_prices(coin)
30
31          # Skip if no data
32          if not prices:
33              print("No data available\n")
34              continue
35
36          returns = calculate_returns(prices)
37
38          volatility = calculate_volatility(returns)
39
40          risk_level = classify_risk(volatility)
41
42          print("Volatility:", round(volatility,6))
43
44          print("Risk Level:", risk_level)
45
46          print("----- \n")
47
48          # Prevent API rate limit
49          time.sleep(5)
50
51
52  # -----
53
54  if __name__ == "__main__":
55      main()
```

- email_alert.py

```

1  import smtplib
2  from email.mime.text import MIMEText
3  from risk import (
4      fetch_crypto_prices,
5      calculate_returns,
6      calculate_volatility
7  )
8
9  # Enter your details here
10 SENDER_EMAIL = "geethavalluri09@gmail.com"
11 SENDER_PASSWORD = "riro ktme xhto oaxb"
12 RECEIVER_EMAIL = "geethavalluri09@gmail.com"
13
14 VOLATILITY_THRESHOLD = 0.005 # Change as needed
15
16
17 def send_email_alert(subject, message):
18     msg = MIMEText(message)
19     msg["Subject"] = subject
20     msg["From"] = SENDER_EMAIL
21     msg["To"] = RECEIVER_EMAIL
22
23     with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
24         server.login(SENDER_EMAIL, SENDER_PASSWORD)
25         server.send_message(msg)
26
27     print(" Alert email sent successfully!")
28
29
30 def check_risk_and_alert(coin):
31     prices = fetch_crypto_prices(coin)
32     returns = calculate_returns(prices)
33     volatility = calculate_volatility(returns)
34
35     print(f"{coin.capitalize()} Volatility: {volatility:.5f}")
36
37     if volatility > VOLATILITY_THRESHOLD:
38         subject = f" High Risk Alert: {coin.capitalize()}"
39         message = f"{coin.capitalize()} volatility is high: {volatility:.5f}"
40         send_email_alert(subject, message)
41     else:
42         print(" Risk level is normal.")
43
44
45 if __name__ == "__main__":
46     check_risk_and_alert("cardano")

```

- prediction.py

```

1  import numpy as np
2  from sklearn.linear_model import LinearRegression
3
4
5  # -----
6  # LINEAR REGRESSION PRICE PREDICTION
7  # -----
8
9  def predict_next_price(prices):
10     """
11     Predict next day's price using Linear Regression
12     """
13
14     prices = np.array(prices)
15
16     # Create day index
17     X = np.arange(len(prices)).reshape(-1, 1)
18
19     # Target variable
20     y = prices
21
22     # Create and train model
23     model = LinearRegression()
24     model.fit(X, y)
25
26     # Predict next day
27     next_day = np.array([[len(prices)]]).reshape(-1, 1)
28     prediction = model.predict(next_day)
29
30     return prediction[0]

```

- app.py

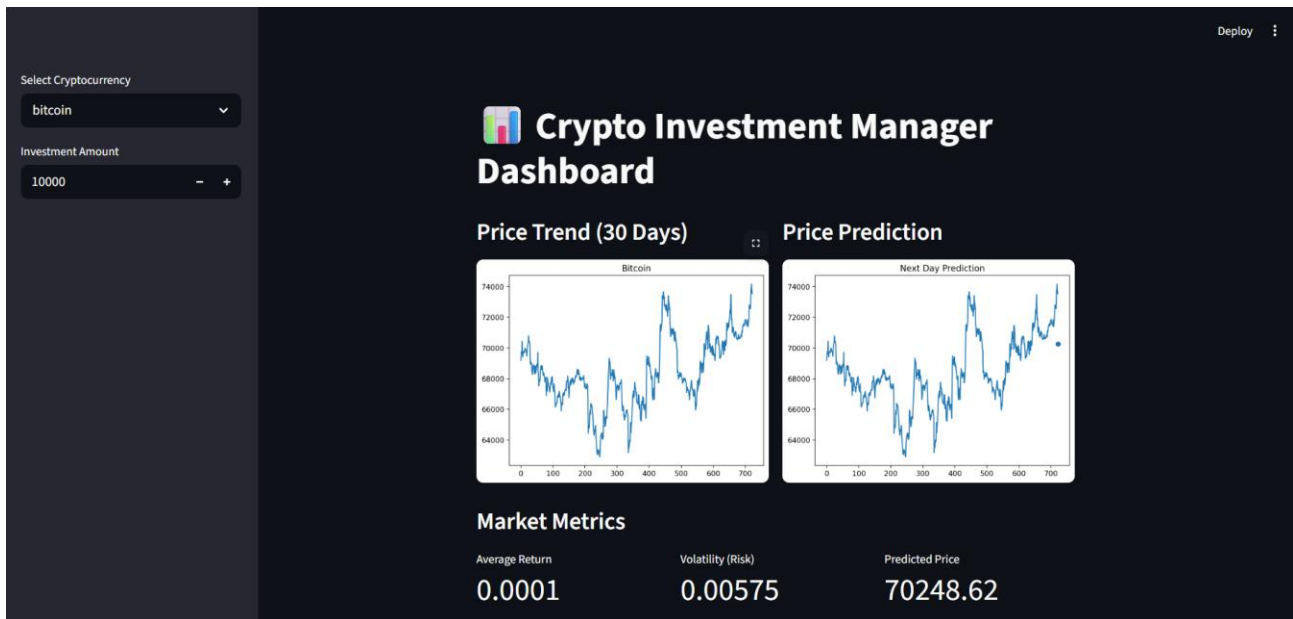
```

1
2
3 # -----
4 # Streamlit Button
5 # -----
6
7 if st.button("Run Crypto Analysis"):
8
9     volatility_data = {}
10    sharpe_data = {}
11    predictions = {}
12
13    for coin in COINS:
14
15        prices = fetch_historical_prices(coin)
16        returns = calculate_returns(prices)
17        exp_return = calculate_expected_return(returns)
18        vol = calculate_volatility(returns)
19        sharpe = sharpe_ratio(exp_return, vol)
20        pred, model = predict_next_price(prices)
21        volatility_data[coin] = vol
22        sharpe_data[coin] = sharpe
23        predictions[coin] = pred
24
25        st.subheader(coin.upper())
26
27        st.write("Expected Return:", round(exp_return,6))
28        st.write("Volatility:", round(vol,6))
29        st.write("Sharpe Ratio:", round(sharpe,4))
30        st.write("Predicted Next Price:", round(pred,2))
31
32        # Price trend chart
33        fig, ax = plt.subplots()
34        ax.plot(prices)
35        ax.set_title(f"{coin} Price Trend")
36        st.pyplot(fig)
37        st.write("Predicted Next Price:", round(pred,2))
38        prediction_fig = plot_prediction(prices, model)
39        st.pyplot(prediction_fig)
40
41
42    # -----
43    # Portfolio Allocation
44    # -----
45
46    sorted_coins = sorted(volatility_data.items(), key=lambda x: x[1])
47    low = [c[0] for c in sorted_coins[:2]]
48    medium = [c[0] for c in sorted_coins[2:4]]
49    high = [c[0] for c in sorted_coins[4:]]
50
51    allocation = {}
52
53    for coin in low:
54        allocation[coin] = TOTAL_INVESTMENT * 0.5 / len(low)
55
56    for coin in medium:
57        allocation[coin] = TOTAL_INVESTMENT * 0.3 / len(medium)
58
59    for coin in high:
60        allocation[coin] = TOTAL_INVESTMENT * 0.2 / len(high)
61
62    st.header("Portfolio Allocation (₹10,000)")
63
64    for coin, amount in allocation.items():
65        st.write(f"{coin} → ₹{round(amount,2)}")
66
67
68    # Pie Chart
69    fig2, ax2 = plt.subplots()
70    ax2.pie(allocation.values(), labels=allocation.keys(), autopct="%1.1f%%")
71    st.pyplot(fig2)

```

7.5.SYSTEM OUTPUT:WEB APPLICATION INTERFACE

Crypto Investment Manager project, the Web Application Interface refers to the user interface where users interact with the system to analyze and invest in cryptocurrencies. The web application interface is developed using Streamlit. It allows users to select cryptocurrencies, view price trends, analyze risk, and see predicted prices. The interface also displays portfolio allocation and generates an investment report based on the analysis.

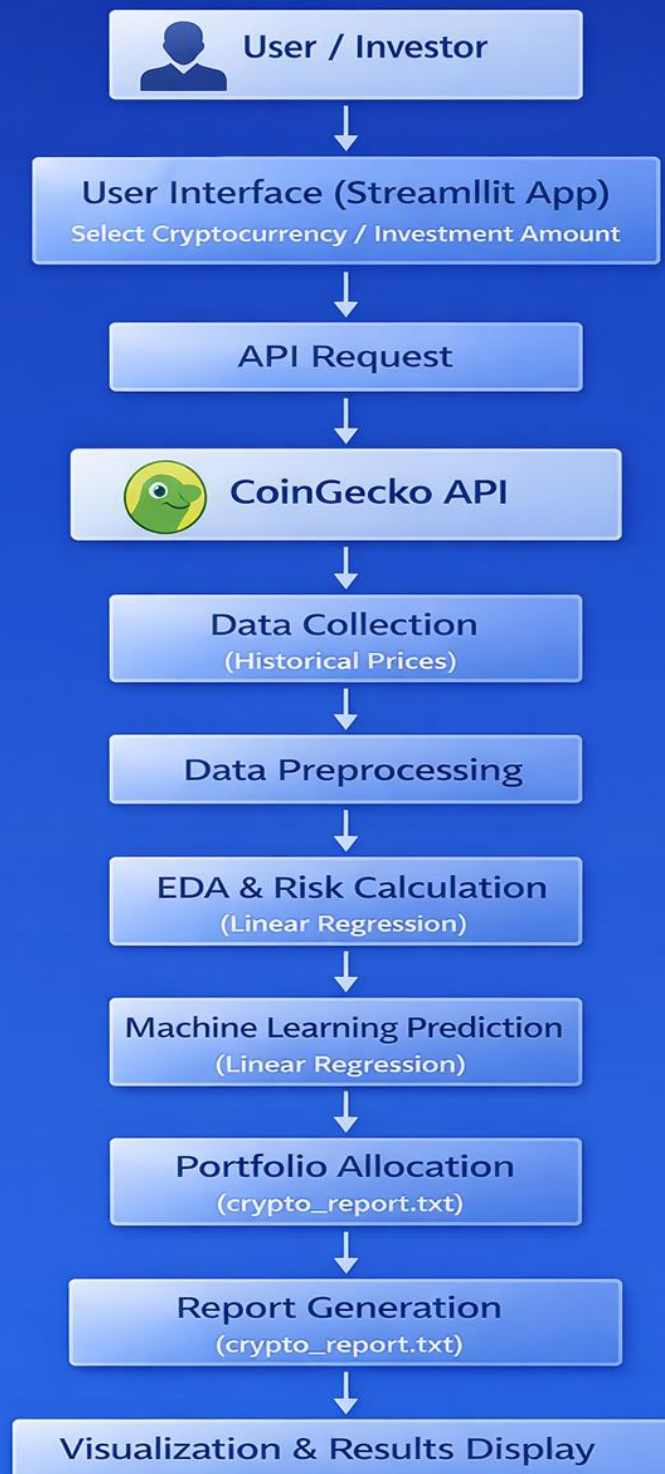


DATA FLOW DIAGRAMS

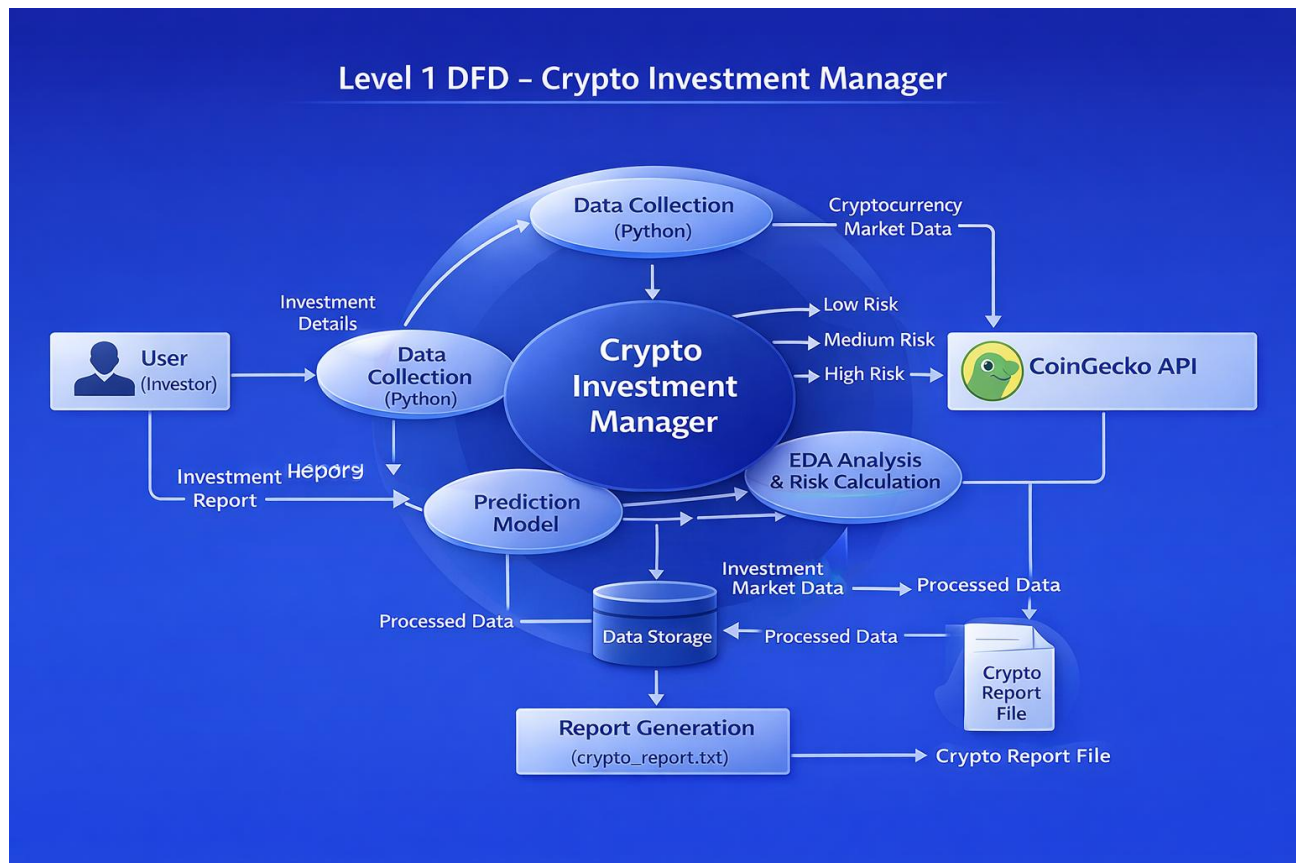
8.DATA FLOW DIAGRAMS

8.1.Level0 DFD:

Crypto Investment Manager – System Flow Diagram



8.2.Level1 DFD:



A **DataFlowDiagram(DFD)** is a visual representation of how data moves through a system. It shows how data enters, how it's processed, and how it exits, without focusing on the system's internal technical details (like code or hardware).

DFDs are typically created at multiple levels to represent increasing detail:

Level0 DFD:

Level 0 DFD is the highest abstraction level, also called the Context Diagram. It shows the entire system as a single process and its interactions with external entities (e.g., users, databases, APIs).

“Level 0 DFD represents the overall interaction of the Crypto Investment Manager system with external entities. The user provides cryptocurrency selection and investment details to the system. The system fetches market data from the CoinGecko API, performs analysis and prediction, and finally generates an investment report with risk analysis and portfolio recommendations.”

Level1DFD(DecompositionDiagram)

Level 1 DFD breaks down the main process from Level 0 into sub-processes, showing more detailed data flow between components. It helps to understand how the system works internally.

“Level 1 DFD shows the internal processes of the Crypto Investment Manager system. The system first collects cryptocurrency data from the CoinGecko API, then preprocesses the data for analysis. Risk metrics and market trends are calculated using EDA techniques. A machine learning model predicts future prices, and based on the risk level, the system allocates investment. Finally, a report is generated showing risk analysis and portfolio recommendations.”

FUTURE ENHANCEMENTS

9.FUTURE ENHANCEMENTS

The current system performs cryptocurrency data analysis, risk evaluation, portfolio allocation, and price prediction using machine learning techniques. However, the project can be further improved by adding advanced features and improving prediction accuracy. Future improvements can focus on integrating advanced machine learning models, expanding the number of cryptocurrencies analyzed, and enhancing the user interface. These enhancements will make the system more efficient, accurate, and useful for real-world cryptocurrency investment analysis.

Possible Future Enhancements

- Implement advanced prediction models such as Long Short-Term Memory (LSTM) for better time-series forecasting.
- Integrate real-time cryptocurrency data streaming to provide live market analysis.
- Support a larger number of cryptocurrencies instead of analyzing only a few selected coins.
- Develop a more interactive and advanced dashboard interface for better user experience.
- Add automated alert systems to notify users about high-risk market conditions or major price changes.
- Implement portfolio optimization algorithms to recommend better investment strategies.
- Allow users to customize investment preferences and risk levels.
- Store historical analysis data using a database system for long-term analysis.
- Enhance visualization with more interactive charts and analytics tools.

RESULTS AND ACHIEVEMENTS

10.RESULTS AND ACHIEVEMNETS

The Crypto Investment Manager system was successfully developed to analyze cryptocurrency market data and assist users in making informed investment decisions. The system collects historical cryptocurrency price data using the CoinGecko API and processes it to perform various financial analyses such as return calculation, volatility measurement, and risk classification. Through exploratory data analysis and visualization techniques, the system helps users understand market trends and price behavior. The project also implements a machine learning model using Linear Regression to predict future cryptocurrency prices based on historical data.

Additionally, the system recommends a portfolio allocation strategy based on the calculated risk levels of different cryptocurrencies. The results are presented through graphical visualizations and a generated report that summarizes the analysis. Overall, the project demonstrates the practical use of data analytics and machine learning techniques in financial market analysis and investment decision support.

Achievements

- Successfully integrated the CoinGecko API to collect real-time and historical cryptocurrency market data.
- Implemented data analysis techniques to calculate returns, volatility, and risk levels.
- Developed a machine learning prediction model using Linear Regression to forecast future cryptocurrency prices.
- Performed exploratory data analysis (EDA) to analyze market trends and patterns.
- Generated visualizations such as price trend graphs, volatility comparison charts, and portfolio allocation charts.
- Implemented a portfolio allocation strategy to distribute investment based on risk categories.
- Built a web-based interface using Streamlit for interactive data visualization.
- Automatically generated an analysis report file (crypto_report.txt) summarizing the results.

CONCLUSION

11.CONCLUSION

The **Crypto Investment Manager** project was successfully developed to analyze cryptocurrency market data and assist users in making informed investment decisions. The system collects historical price data using the CoinGecko API and processes it to perform various financial analyses such as return calculation, volatility measurement, and risk classification. Through exploratory data analysis and visualization techniques, the project provides insights into market trends and cryptocurrency price behavior.

The implementation of a prediction model using Linear Regression allows the system to forecast future cryptocurrency prices based on historical data. In addition, the system recommends a portfolio allocation strategy by categorizing cryptocurrencies into different risk levels, helping users understand how to distribute their investments effectively.

The development of a user-friendly dashboard using Streamlit enables users to interact with the system, view visualizations, and analyze cryptocurrency performance easily. Overall, the project demonstrates how data analytics and machine learning techniques can be applied to financial markets to support better investment planning and decision-making.

REFERENCES

12.REFERENCES

1. Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

<https://matplotlib.org>

2. Streamlit – Python Framework for Data Applications.

Streamlit is an open-source Streamlit Python library for building data-driven web apps with minimal code. It is widely used by data scientists, ML engineers, and analysts to turn scripts into interactive dashboards without needing traditional front-end development.

<https://streamlit.io/>

3. Python:<https://www.w3schools.com/python/>

4. Numpy:<https://www.w3schools.com/python/numpy/default.asp>

5. Scikit-learn:<https://scikit-learn.org/stable/>

6. CoinGecko API – Cryptocurrency Market Data API Documentation.

The CoinGecko API is a web-based cryptocurrency data service that exposes REST and WebSocket endpoints for accessing real-time and historical data on coins, tokens, NFTs, exchanges, and on-chain DEX activity. It's widely used in trading apps, wallets, analytics dashboards, and other Web3 tools to fetch prices, market stats, and metadata programmatically.

https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart?vs_currency=usd&days=30



ANDHRA PRADESH STATE COUNCIL OF HIGHER EDUCATION

(A Statutory Body of the Government of Andhra Pradesh)

2nd, 3rd, 4th and 5th floors, Neeladri Towers, Sri Ram Nagar, 6th Battalion Road
Atmakur (V)Mangalagiri (M), Guntur, Andhra Pradesh, Pin - 522 503
www.apsche.ap.gov.in